

Space-Partitioned ND-Trees for the Dynamic Nondominance Problem

Bruno Lang 

Abstract—We present techniques for improving the efficiency of ND-trees in the solution of the dynamic nondominance problem, i.e., for building and maintaining a Pareto archive. We propose algorithms for (re)building a tree from a set of nondominated points, either as a space-partitioned ND-tree or by a clustering approach. Numerical experiments confirm that rebuilding the tree at intervals, combined with modified strategies for updating the lower and upper bounds in each node and for computing the distances of points to subtrees in between the rebuilds, can lead to significant speedups over using plain ND-tree-based updates throughout, in particular for higher dimensions.

Index Terms—Dynamic nondominance problem, multiobjective optimization.

I. INTRODUCTION

IN MULTI-OBJECTIVE optimization, one tries to maximize (or minimize) D objective functions $f_k(\mathbf{x})$, $k = 1, \dots, D$, subject to constraints $g_j(\mathbf{x}) \leq 0$ (or $= 0$), $j = 1, \dots, m$, with $m \geq 0$ and $\mathbf{x} \in \mathbb{R}^q$. Typically, the objectives are conflicting, such that there is no single design $\mathbf{x}^*$ optimizing all objectives. Then, one resorts to the concept of Pareto optimality.

Consider a set $\mathcal{Y}$ of objective (or outcome) vectors, $\mathcal{Y} = \{\mathbf{y} = \mathbf{f}(\mathbf{x}) \equiv (f_1(\mathbf{x}), \dots, f_D(\mathbf{x})) : \mathbf{x} \in \mathcal{X}\}$, corresponding to a set $\mathcal{X} \subseteq \mathbb{R}^q$ of feasible designs, i.e., $g_j(\mathbf{x}) \leq 0$ for all $\mathbf{x} \in \mathcal{X}$ and $j = 1, \dots, m$. Then, $\mathbf{y} \in \mathcal{Y}$ is said to be *weakly dominated* by $\tilde{\mathbf{y}} \in \mathcal{Y}$, $\mathbf{y} \preceq \tilde{\mathbf{y}}$, if

$$\mathbf{y}(k) \leq \tilde{\mathbf{y}}(k) \quad \text{for all } k = 1, \dots, D \quad (1)$$

where $\mathbf{y}(k) \in \mathbb{R}$ is the k th component of $\mathbf{y}$. $\mathbf{y}$ is (strictly) *dominated* by $\tilde{\mathbf{y}}$, $\mathbf{y} \prec \tilde{\mathbf{y}}$, if in addition $\mathbf{y} \neq \tilde{\mathbf{y}}$, i.e., strict inequality holds in at least one component. If all vectors in $\mathcal{Y}$ are mutually nondominated ($\mathbf{y} \not\prec \tilde{\mathbf{y}}$ for all $\mathbf{y} \neq \tilde{\mathbf{y}}$) then none of the objective values can be increased without decreasing another one. Note that different designs $\mathbf{x}$ may lead to the same nondominated $\mathbf{y}$.

Once a set of points with nondominated objectives has been determined, one or more of them can be selected according to additional criteria. Ideally one would like to know the full

Pareto front, that is, the nondominated $\mathbf{y}$ w.r.t. to *all* feasible $\mathbf{x}$. In practice, one has to approximate it by a finite set.

When referring to “points” $\mathbf{p}$ in the remainder of this work we will always identify $\mathbf{p} \equiv \mathbf{y}$, because for the following discussions, the design variables $\mathbf{x}$ can be ignored. In practice, however, the design variables are the ones of true interest, and the objectives are mainly used for determining dominance; then one would store both $\mathbf{y}$ and $\mathbf{x}$ in $\mathbf{p}$.

The nondominated points are also called the *maxima* of the point set; if we replace “ $\leq$ ” with “ $\geq$ ” in (1) then they will be the *minima* of the set.

Dominated points may be removed either *a posteriori*, when all potential “candidate points” $\mathbf{p}_i$ are known, or during their generation, leading to the following two problems.

The *static nondominance problem* consists of determining, for a given set $\mathcal{P} = \{\mathbf{p}_1, \dots, \mathbf{p}_n\}$, its nondominated points

$$\mathcal{P}^N = \{\mathbf{p}_\ell \in \mathcal{P} : \nexists \mathbf{p}_j \in \mathcal{P} : \mathbf{p}_\ell \prec \mathbf{p}_j\}.$$

The *dynamic nondominance problem* addresses maintaining a *Pareto archive* when adding points, i.e., determining

$$\mathcal{P}_i^N = \{\mathbf{p}_\ell \in \mathcal{P}_i : \nexists \mathbf{p}_j \in \mathcal{P}_i : \mathbf{p}_\ell \prec \mathbf{p}_j\}$$

for $i = 1, 2, \dots$, where $\mathcal{P}_i = \{\mathbf{p}_1, \dots, \mathbf{p}_i\}$. Since $\mathcal{P}_i^N$ reflects only dominance among the first i points, a point in $\mathcal{P}_i^N$ may be dominated by a later $\mathbf{p}_j$ and then be removed from the archive.

The dynamic nondominance problem arises naturally in multiobjective evolutionary algorithms (MOEAs) [1]. These algorithms approximate the Pareto front with a finite set. They propagate a *population* of designs over time, obtaining the next *generation* from the previous one via rules inspired by biological systems, such as selection, crossover, and mutation. In addition to the current population, they often rely on an archive, either to just save “good” points from being forgotten or to also have a larger pool for selection. Dominated points can be removed from this archive, making it a Pareto archive.

There are many variants of the general evolutionary approach; cf. [2]–[7] for just a few of them. The contribution of this work is not in advancing MOEA methodology as such, but in making the maintenance of a Pareto archive more efficient. Algorithm 1 summarizes the basic procedure for this task.

The obvious approach for lines 3 and 4 would compare $\mathbf{p}_i$ to all $\mathbf{p}_j \in \mathcal{P}^N$. This *linear search* is indeed used in some MOEA variants, and it is efficient if the archive is kept small by also removing selected nondominated points. This, however, can impair the quality of the attainable approximation to the overall Pareto front; cf. [8].

Manuscript received 28 May 2021; revised 21 October 2021 and 5 January 2022; accepted 14 January 2022. Date of publication 25 January 2022; date of current version 3 October 2022.

The author is with the IMACM and Scientific Computing, Bergische Universität Wuppertal, 42097 Wuppertal, Germany (e-mail: lang@math.uni-wuppertal.de).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TEVC.2022.3145631>.

Digital Object Identifier 10.1109/TEVC.2022.3145631

Algorithm 1: Maintaining a Pareto Archive

Input : Points $\mathbf{p}_1, \dots, \mathbf{p}_n$
Output: Pareto archive $\mathcal{P}_1^N, \dots, \mathcal{P}_n^N$ for $i = 1, \dots, n$

```

1  $\mathcal{P}^N = \emptyset$ 
2 for  $i = 1, \dots, n$ 
3   if  $\mathbf{p}_i$  is not dominated by any point in  $\mathcal{P}^N$  then
4     remove all points  $\mathbf{p}_j$  from  $\mathcal{P}^N$  that are dominated
       by  $\mathbf{p}_i$ 
5     insert  $\mathbf{p}_i$  into  $\mathcal{P}^N$ 
  // now  $\mathcal{P}^N$  is  $\mathcal{P}_i^N$ 

```

For unconstrained archives, several data structures have been proposed over the last decades to avoid linear search by making better use of the points' positions, including Pareto-Optimal Trees [9], Quad-trees [10], [11], Dominance Decision Trees [12], Dominated and Nondominated Trees [8], M-front [13], Binary Space Partitioning Trees [14], and ND-trees [15], to name just a few. In benchmarks, ND-trees have proved a particularly efficient tool [15], [16]. In this work, we propose techniques for further improving their performance.

The remainder of this article is organized as follows. To ease the ensuing discussion, we first briefly review ND-trees and their algorithms in Section II. In Section III, we introduce space-partitioned ND-trees, which are particular ND-trees that can be built efficiently from a set of nondominated points and can make ND-tree-based nondominance checks more effective. An alternative way for building an ND-tree from nondominated points is presented in Section IV; it is closer to the strategy for adding points to an ND-tree in [15]. In Section V, we discuss how the ability to efficiently (re)build an ND-tree can be used and combined with other modifications to the original ND-tree algorithms to improve overall efficiency. The numerical experiments reported in Section VI show that rebuilding the tree from time to time can significantly reduce the overall time for maintaining a large Pareto archive. We summarize our findings in Section VII.

II. ND-TREES

The description in this section mainly follows [15]. Recall that a *tree* consists of *nodes*, each of which is connected to one *parent* node (except for one node, the *root*, which has no parent) and to (up to some fixed number of) *child* nodes. Since many operations with trees are recursive, progressing from the current node to its children, their description is simplified by distinguishing *leaf* nodes (those having no children) and *non-leaf* nodes (at least one child); cf. [17] for a deeper treatment of trees. An ND-tree stores the (nondominated) points of $\mathcal{P}_i^N$ as follows.

- 1) The points are stored only in the leaf nodes, with up to P points per leaf. Nonleaf nodes have up to C children. P and C can be chosen freely such that $C \leq P + 1$; Jaskiewicz and Lust [15] considered dimensions $D \leq 10$ and found $C = D + 1$ and $P = 20$ to yield adequate performance.

- 2) Each node v represents a set $\mathcal{S}_v$ of points. For a leaf node, $\mathcal{S}_v$ comprises just the points stored in the node, and for a nonleaf node, $\mathcal{S}_v$ contains all points represented by the children. That is, $\mathcal{S}_v$ represents all points that are stored somewhere in the subtree rooted at v .
- 3) In each node v , a lower bound $\mathbf{lb}$ and an upper bound $\mathbf{ub}$ for its set $\mathcal{S}_v$ is stored.

Here, $\mathbf{lb} \in \mathbb{R}^D$ is a *lower bound* for a (finite) set $\mathcal{S} = \{\mathbf{p}_1, \dots, \mathbf{p}_m\}$ if $\mathbf{lb} \preceq \mathbf{p}_j$ for all $j = 1, \dots, m$. The *infimum* of $\mathcal{S}$, $\inf(\mathcal{S})$, is its greatest lower bound. *Upper bounds* $\mathbf{ub}$ and the *supremum* $\sup(\mathcal{S})$ are defined accordingly. Note that $\inf(\mathcal{S})$ and $\sup(\mathcal{S})$ are easily obtained with $\mathcal{O}(Dm)$ operations by taking the smallest and largest value $\mathbf{p}_j(k)$ in each component k . Thus, they describe the set's *bounding box* [18] $\mathbf{BB}(\mathcal{S}) = \{\mathbf{y} \in \mathbb{R}^D : \inf(\mathcal{S}) \preceq \mathbf{y} \preceq \sup(\mathcal{S})\}$. The supremum (or an upper bound) is also known as (*approximate*) *ideal point*, and the infimum (or a lower bound) is also called (*approximate*) *nadir* [15].

The efficiency of ND-tree-based updates $\mathcal{P}_{i-1}^N \rightarrow \mathcal{P}_i^N$ with a new point $\mathbf{p}_i$ is based on the following observations. If, in some node v in the tree, $\mathbf{p}_i \prec \mathbf{lb}(\mathcal{S}_v)$ then $\mathbf{p}_i$ is dominated by all points in $\mathcal{S}_v$ and, therefore, $\mathbf{p}_i$ is discarded (case 1). Otherwise, if $\mathbf{ub}(\mathcal{S}_v) \prec \mathbf{p}_i$, then $\mathbf{p}_i$ dominates all points in $\mathcal{S}_v$ and the whole subtree v is removed (case 2). Otherwise, if $\mathbf{p}_i \not\preceq \mathbf{ub}(\mathcal{S}_v)$ and $\mathbf{lb}(\mathcal{S}_v) \not\preceq \mathbf{p}_i$ (case 3) then there is at least one component k such that $\mathbf{p}_i(k) > \mathbf{ub}(\mathcal{S}_v)(k)$ and thus $\mathbf{p}_i(k) > \mathbf{p}_j(k)$ for all $\mathbf{p}_j \in \mathcal{S}_v$, and there is a component k' such that $\mathbf{lb}(\mathcal{S}_v)(k') > \mathbf{p}_i(k')$ and thus $\mathbf{p}_j(k') > \mathbf{p}_i(k')$ for all $\mathbf{p}_j \in \mathcal{S}_v$, which implies that no point in $\mathcal{S}_v$ can dominate $\mathbf{p}_i$ and $\mathbf{p}_i$ dominates none of these points. Therefore, we do not need to consider the subtree v further. If none of the above applies, we are in case 4, and we either compare $\mathbf{p}_i$ to all points stored in v (if v is a leaf) or recursively process v 's children. This procedure is summarized in Algorithm 2 (essentially Algorithm `UpdateNode` from [15], except that we require strict instead of weak dominance for cases 1 and 2 to allow for multiple points).

Calling `NonDomPrune` with the root of $\mathcal{P}_{i-1}^N$ covers lines 3 and 4 of Algorithm 1. If this call returns `TRUE` then $\mathbf{p}_i$ must be inserted into the (pruned) tree, which is done by calling the function `Insert` shown in Algorithm 3 (also from [15]).

In general, the nondominance check and pruning will by far dominate the ensuing `Insert` in terms of operations and time because many nodes will be visited due to the recursive calls in case 4 of `NonDomPrune`. We may hope to keep the number of these calls low if the boxes $[\mathbf{lb}, \mathbf{ub}]$ of the children in each node are small and do not overlap very much, i.e., the assignment of the points to the subtrees reflects geometric locality. This clustering drives the decision where new points are placed in `Insert` and how overflowing leaves are split.

We do not repeat the functions for splitting an overflowing leaf node and for widening the bounds, but only summarize their workings.

`SplitLeaf` changes an overflowing leaf node into a nonleaf node with C new leaf nodes as children and distributes the $P + 1$ points to these. It first selects C *seed points* from $\mathcal{S}_v$, each of which is placed in a different child. The first seed point is chosen to have the largest average distance to all other points

Algorithm 2: NonDomPrune(v, p_i)—Nondominance Check and Pruning

Input : Tree with root node v , additional point p_i
Output: FALSE if p_i is dominated by any point in the subtree v , and TRUE otherwise;
in addition, all points that are dominated by p_i are removed from the tree v

```

1 if  $p_i < \text{lb}(\mathcal{S}_v)$  then // Case 1
2   return FALSE
3 else if  $\text{ub}(\mathcal{S}_v) < p_i$  then // Case 2
4   delete subtree  $v$  and return TRUE
5 else if  $p_i \preceq \text{ub}(\mathcal{S}_v)$  or  $\text{lb}(\mathcal{S}_v) \preceq p_i$  then // Case 4
6   if  $v$  is a leaf node then
7     for all  $p_j$  stored in  $v$ 
8       if  $p_i < p_j$  then
9         return FALSE
10      else if  $p_j < p_i$  then
11        delete  $p_j$  from  $v$ 
12   else
13     for all children  $\gamma$  of  $v$ 
14       if NonDomPrune( $\gamma, p_i$ ) = FALSE then
15         return FALSE
16       else if  $\gamma$  became empty then
17         remove  $\gamma$ 
18     if only one child  $\gamma$  of  $v$  remained then
19       replace  $v$  with  $\gamma$ 
20   return TRUE
21 else // Case 3
22   return TRUE

```

Algorithm 3: Insert(v, p_i)—Insert a Point Into a Tree

Input : Tree with root node v ,
additional point p_i
Output: Tree including the point p_i

```

1 if  $v$  is a leaf then
2   add  $p_i$  to  $\mathcal{S}_v$  (i.e., store the point in this node)
3   WidenBounds( $v, p_i$ )
4   if  $v$  overflows ( $P + 1$  points in  $\mathcal{S}_v$ ) then
5     SplitLeaf( $v$ )
6 else
7   Insert( $\gamma, p_i$ ), where  $\gamma$  is the child of  $v$  that is closest to  $p_i$ 

```

in $\mathcal{S}_v$, and the remaining seed points are chosen successively to maximize the average distance to all prior seed points. Finally, each of the remaining $P + 1 - C$ nonseed points is assigned to the closest child. Here, as well as in line 7 of Algorithm 3, the distance of a point p to a child γ (i.e., a whole subtree) is taken to be the distance of p to the center of the subtree's box,

$(\text{lb}(\gamma) + \text{ub}(\gamma))/2$, and this box includes all points inserted in that subtree so far.

Updating the bounds is done with **WidenBounds**, which adjusts lb and ub to also cover the additional point p_i and, if one of the bounds has changed, also forces the update of the parent's bounds. This recursion may progress up to the root of the whole tree $\mathcal{P}^N$.

This way of updating is successful in maintaining the tree reasonably well balanced in many cases, but it has already been pointed out in [15] that in extreme cases, a completely unbalanced tree may be built. In practice, two effects may reduce the effectiveness of the nondominance tests over time. First, in order to minimize “effort per node,” the bounds lb and ub are *not tightened* if points or subtrees are deleted. Second, the balancing tends to deteriorate slightly over time (e.g., nonleaf nodes may lose some of their C children due to pruning).

To counteract the latter effect, we have tried different approaches for *maintaining* balance during the updates. From our experience, local rebalancing cannot be recommended because it can interfere too much with the beneficial geometrical clustering and does not improve performance consistently. To give an example, consider the situation where a nonleaf node loses one or several children during **NonDomPrune** because they became empty. The full number of C children can be restored by (repeatedly) “lifting” the grandchild η with the largest $\mathcal{S}_\eta$. While this strategy was able to reduce the time for building a Pareto archive from 64 Ki = 65 536 CUBE-distributed 10-D points from 4.34 to 4.17 s, for a second set with identically distributed (but different) points the time increased from 3.78 to 4.08 s. (More details on the test sets will be given in Section VI.)

Instead, we propose to enforce a suitable structure by completely *rebuilding* the tree after some number of steps, as pointed out in the following section.

III. SPACE-PARTITIONED ND-TREES

Our technique is based on the observations that:

- 1) the ND-tree-update with **NonDomPrune** and **Insert** can be done whenever lower and upper bounds are available in each node of the tree, *no matter how this tree had been built*;
- 2) if the points are already known to be mutually nondominated then nondominance checks are not necessary when building the tree;
- 3) building a tree with almost perfect balancing (w.r.t. both height and number of points) and no (geometrical) overlap of the children is simple.

The approach is similar to the well-known multidimensional binary search tree [19], [20], except that we do not assume a *binary* tree and store the points exclusively at the leaves, keeping only the lb and ub bounds in nonleaf nodes.

Fig. 1 shows how $n = 17$ points would be assigned to $C = 4$ (left picture) or $C = 3$ (right) children of a node (note that since we assume the points to be mutually nondominated, this must be a 2-D projection of higher-dimensional points), Algorithms 4 and 5 give the details. The whole tree is built by calling **SPNDBuild**($p_1, \dots, p_n, 1$).

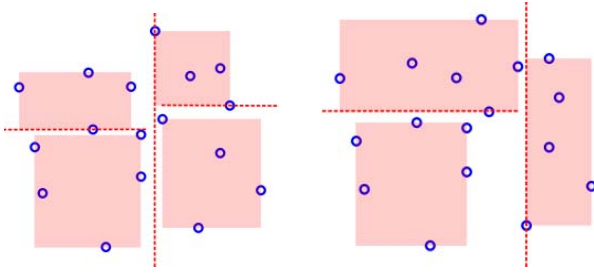


Fig. 1. SPND partitioning of 17 points for $C = 4$ (left) and $C = 3$ (right). (2-D projections of higher-dimensional points.)

Algorithm 4: SPNDBuild($\mathbf{p}_1, \dots, \mathbf{p}_m, k$)—Build SPND-tree

Input : Nondominated points $\mathbf{p}_1, \dots, \mathbf{p}_m$,
direction $k \in \{1, \dots, D\}$

Output: Space-partitioned ND-tree ν containing these points

```

1 generate a new node  $\nu$ 
2 if  $m \leq P$  then
    // all points fit into one node
3   make  $\nu$  a leaf node and insert all  $\mathbf{p}_i$  into its  $S_\nu$ 
4 else
5   make  $\nu$  a non-leaf node with  $C$  children  $\gamma_1, \dots, \gamma_C$ 
6   DistributePoints( $\mathbf{p}_1, \dots, \mathbf{p}_m, \gamma_1, \dots, \gamma_C, k$ )
7 ComputeBounds( $\nu$ )

```

Algorithm 5: DistributePoints($\mathbf{p}_1, \dots, \mathbf{p}_m, \gamma_{\text{first}}, \dots, \gamma_{\text{last}}, k$)—Distribute the Points $\mathbf{p}_1, \dots, \mathbf{p}_m$ to the Children $\gamma_{\text{first}}, \dots, \gamma_{\text{last}}$ and Start by Slicing Along Direction k

Input : Nondominated points $\mathbf{p}_1, \dots, \mathbf{p}_m$,
range of children $\gamma_{\text{first}}, \dots, \gamma_{\text{last}}$,
direction $k \in \{1, \dots, D\}$

```

1 if first = last then
    // generate subtree for this child
2    $\gamma_{\text{first}} = \text{SPNDBuild}(\mathbf{p}_1, \dots, \mathbf{p}_m, k)$ 
3 else
    // at least two children:
    // make two groups
    // size of first group:
4    $\text{children1} = \lfloor (\text{last} - \text{first} + 1) / 2 \rfloor$ 
    // last child in first group:
5    $\text{last1} = \text{first} + \text{children1} - 1$ 
    // points for first group:
6    $\text{points1} = \lfloor (m \cdot \text{children1}) / (\text{last} - \text{first} + 1) \rfloor$ 
7   PartitionAtPosition( $\mathbf{p}_1, \dots, \mathbf{p}_m, \text{points1}, k$ )
8   DistributePoints( $\mathbf{p}_1, \dots, \mathbf{p}_{\text{points1}},$ 
         $\gamma_{\text{first}}, \dots, \gamma_{\text{last1}}, k + 1$ )
9   DistributePoints( $\mathbf{p}_{\text{points1}+1}, \dots, \mathbf{p}_m,$ 
         $\gamma_{\text{last1}+1}, \dots, \gamma_{\text{last}}, k + 1$ )

```

For $n = 17$ and $C = 4$, this initial call first generates a nonleaf node with four children and calls DistributePoints($\mathbf{p}_1, \dots, \mathbf{p}_{17}, \gamma_1, \dots, \gamma_4, 1$) to distribute the

points to these. This call makes two groups $\{\gamma_1, \gamma_2\}$ and $\{\gamma_3, \gamma_4\}$ from the children and assigns 8 and 9 points to these (this is done in the recursive calls). The $\text{points1} = 8$ points for the first group are chosen to be the largest ones w.r.t. the $(k = 1)$ st direction. To this end, we determine a point with points1 -largest value $\mathbf{p}(k)$ among the $\mathbf{p}_i$ and *rear-range* the sequence such that this point is placed at position $\mathbf{p}_{\text{points1}}$ and all points preceding it (following it) have the k th coordinate larger or equal (smaller or equal) to this value. Therefore, the first points1 points of the rearranged sequence are assigned to the first group. It is well known [17] that, for any $j \in \{1, \dots, m\}$, the j -largest entry (such as the median) can be determined and the sequence be reordered with $\mathcal{O}(m)$ comparisons, without sorting, and our PartitionAtPosition implements an efficient variant of this approach.

The two recursive calls further subdivide the groups and assign points to the subgroups, slicing along the second coordinate. The resulting four recursive calls are with a single child each and therefore a new tree is built with the points assigned to it by calling SPNDBuild, slicing along the third dimension. (The dimensions are used cyclically, i.e., $D + 1$ is replaced with 1.)

For $C = 3$, the first call to DistributePoints will generate two groups $\{\gamma_1\}$ and $\{\gamma_2, \gamma_3\}$ and assign one third (two thirds) of the points to these, and again the points of both groups will be further subdivided along the second dimension. (For the second group, this is done within the current node via another level of DistributePoints calls, for the first group, it is done in a new node via SPNDBuild.)

We call the trees generated this way *space-partitioned ND-trees* or *SPND-trees*, due to the way, the points were assigned to the subtrees, and because the trees contain valid (even tight) bounds lb and ub allowing to use them for ND-tree-based updates. They also have the property that the children's boxes may touch one another, but do not overlap.

One might wonder why we do not separate the slicing directions, with 1-D slicing in each node (same direction in all nodes at the same depth in the tree). To give an example, the partitioning in the left picture of Fig. 1 might also correspond to two children (left and right half of the points) and four grandchildren (lower and upper halves of these). However, this increases the number of nodes in the tree, and in our experiments, it also led to higher numbers of visited nodes in NonDomPrune, and since almost all of these lead to case 3 or case 4 requiring four dominance tests, overall time went up as well.

IV. CLUSTERING-BASED BUILDING

Another way of building an ND-tree from a set of nondominated points is by clustering them w.r.t. geometrical proximity, similar to the Insert and SplitLeaf procedures for updating the tree; cf. Section II. This approach is detailed in Algorithm 6.

If the number of points is small enough then all of them are assigned to a new leaf node, and we are done. Otherwise, we generate a new nonleaf node with C children, and these are obtained by grouping the points (w.r.t. distance) into C clusters and recursively building trees from these. We first

Algorithm 6: ClusteredNDBuild($\mathbf{p}_1, \dots, \mathbf{p}_m$)—Build ND-Tree From Nondominated Points

Input : Set $\mathcal{P} = \{\mathbf{p}_1, \dots, \mathbf{p}_m\}$ of nondominated points
Output: ND-tree ν containing these points

```

1 if  $m \leq P$  then
2    $\nu =$  new leaf node containing all points
3 else
4   // find seed points  $\mathbf{s}_\gamma$  for
   //  $C$  clusters
5    $\mathbf{s}_2 = \mathbf{p}_1$ 
6   for iter = 1, ..., 3
7      $(\mathbf{s}_1, \mathbf{s}_2) =$ 
8        $(\mathbf{s}_2, \text{point with largest distance from } \mathbf{s}_2)$ 
9   for  $\gamma = 3, \dots, C$ 
10     $\mathbf{s}_\gamma =$  point with largest
11      (minimum) distance to  $\mathbf{s}_1, \dots, \mathbf{s}_{\gamma-1}$ 
12    // assign the points to the
13    // clusters; try to keep these
14    // reasonably balanced
15    for  $i = 1, \dots, m$ 
16      add  $\mathbf{p}_i$  to cluster  $\gamma$ , where  $\mathbf{s}_\gamma$  is the seed point
17        closest to  $\mathbf{p}_i$ 
18    determine  $\text{size}_{\min}$  and  $\text{size}_{\max}$ , the sizes of the
19      smallest and largest clusters
20    if  $\text{size}_{\max} > 2 \cdot \text{size}_{\min}$  &  $\text{size}_{\max} > m/2$  then
21      do a second pass for determining the seed points
22      (see 1) in main text)
23    bucket sort the points into subsequences  $\mathcal{P}(\gamma)$  for
24      each cluster
25    // post-process sequences to avoid
26    // empty children
27    while one of the  $\mathcal{P}(\gamma)$  is empty do
28      split longest  $\mathcal{P}(\gamma)$  (see 2) in main text)
29    // work at this level almost done;
30    // generate child nodes
31     $\nu =$  new node with children
32      ClusteredNDBuild( $\mathcal{P}(\gamma)$ ),  $\gamma = 1, \dots, C$ 
33 ComputeBounds( $\nu$ )
  
```

determine one *seed point* $\mathbf{s}_\gamma$ for each cluster, $\gamma = 1, \dots, C$. For $\mathbf{s}_1$ and $\mathbf{s}_2$, we select two of the $\mathbf{p}_i$ having “large distance.” This is done similarly to determining *pseudo-peripheral* points in the context of Cuthill–McKey reordering for sparse matrices [21], [22]. Note that finding the maximum distance (i.e., the *diameter* of the point set) would require $\Omega(m \log m)$ operations (cf. [23], also for a fast algorithm); our approach abandons optimality for speed. Then, the remaining seed points $\mathbf{s}_3, \dots, \mathbf{s}_C$ are chosen such that $\mathbf{s}_\gamma$ is “far away” from the previous seed points in the sense that it maximizes the distance to the nearest of the points $\mathbf{s}_1, \dots, \mathbf{s}_{\gamma-1}$. Now, we assign each point $\mathbf{p}_i$ to the cluster γ whose seed point $\mathbf{s}_\gamma$ is closest to $\mathbf{p}_i$, and finally we recursively process the clusters. See Fig. 2 for partitionings obtained with this approach (same points as in Fig. 1).

Two points in the algorithm deserve further explanation.

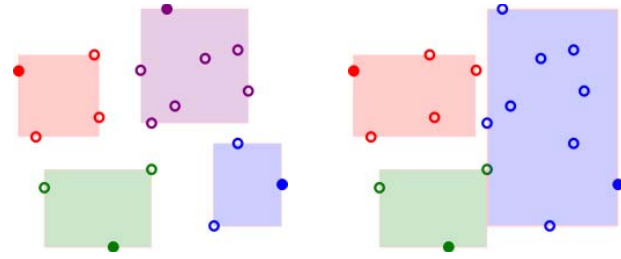


Fig. 2. Clustered ND partitioning of 17 points for $C = 4$ (left) and $C = 3$ (right). Filled circles denote the seed points of the clusters.

- 1) While trees built this way tend to be better balanced than those resulting from just adding the points sequentially, the clusters generated at one node may still differ significantly in size. Therefore, before the clusters are processed further we check if one of them dominates the others, i.e., the largest cluster is more than twice as large as the smallest cluster and contains more than half of the overall points (line 12). If this holds true then we break up the largest cluster. This is done by doing lines 4–11 a second time, but now only points from the largest cluster are considered for choosing $\mathbf{s}_1$ and $\mathbf{s}_2$ in lines 4–6, whereas the remaining seed points are again chosen from all $\mathbf{p}_i$. If the new seed points in fact can reduce $\text{size}_{\max} - \text{size}_{\min}$ then we proceed with these; otherwise, we switch back to the original ones.
- 2) In the presence of multiple points (e.g., with the TRICKY data set described in Section VI), it can happen that fewer than C of the points $\mathbf{p}_i$ are distinct. Then, this also holds for the seed points and, therefore, some clusters γ' remain empty. This is addressed by splitting the longest subsequence $\mathcal{P}_{\text{old}}(\gamma)$ into two subsequences $\mathcal{P}_{\text{new}}(\gamma) \cup \mathcal{P}_{\text{new}}(\gamma')$ until all C subsequences are nonempty.

To improve efficiency, we use one additional size- m real array `minDist` and two size- m integer arrays `nearestSeed` and `perm`. After the `iter` loop in line 5, the distance of each point $\mathbf{p}_i$ from $\mathbf{s}_1$ will be available in `minDist(i)`, and similarly, by just comparing with the distance to the most recent seed point $\mathbf{s}_{\gamma-1}$, `minDist(i)` can be updated to contain the distance of $\mathbf{p}_i$ to the nearest of $\mathbf{s}_1, \dots, \mathbf{s}_{\gamma-1}$ (and `nearestSeed(i)` to contain the number of that seed point) after the γ th iteration of the loop in line 7. Thus, the loop in line 9 requires just one more distance computation (to $\mathbf{s}_C$) per point. Therefore, at most $(3 + 1 + (C - 2) + 1) \cdot m$ distance computations are required per pass. (Note that in the second pass, the distances of *all* $\mathbf{p}_i$ from $\mathbf{s}_1$ must be computed after the `iter` loop.) For fixed C , the overall complexity of `ClusterNDBuild` is $\mathcal{O}(m)$ for the current node (plus recursive calls); `perm` enables an $\mathcal{O}(m)$ in-place bucket sort.

V. EFFICIENCY CONSIDERATIONS

In this section, we discuss several aspects that have an impact on overall efficiency, including how often (and when) we should rebuild the tree, as well as improving the “ND-tree-mode” updates between the rebuilds.

Algorithm 7: Maintaining a Pareto Archive Based on ND-trees and Rebuild

Input : Set $\mathcal{P} = \{\mathbf{p}_1, \dots, \mathbf{p}_n\}$
Output: Tree t containing the nondominated of these points

```

1  $t = \emptyset$ 
2 for  $i = 1, \dots, C$ 
3   if  $\text{NonDomPrune}(t, \mathbf{p}_i)$  then
4      $\text{Insert}(t, \mathbf{p}_i)$ 
5   if condition for rebuilding is met then
6      $\mathcal{P}^N = \text{all points in } t$ 
7      $\text{Delete}(t)$ 
8     // release nodes to a free list
9     build new tree  $t$  from  $\mathcal{P}^N$ 
10    // SPND or clustered

```

A. Rebuilding Strategies

Algorithm 7 summarizes the overall procedure for building a Pareto archive in an ND-tree with intermediate rebuilds.

There are different ways to define the “condition for rebuilding” in line 5.

The simplest strategy is to rebuild on a regular basis, i.e., when $\text{iter}_{\text{rebuild}}$ iterations of the i loop have been done since the previous rebuild. To avoid too frequent rebuilds of large trees, $\text{iter}_{\text{rebuild}}$ is increased with each rebuild. In our experiments, we started with $\text{iter}_{\text{rebuild}} = 512$ and increased it by a factor of 1.5 with each rebuild. Thus, for $n = 1 \text{ Mi} = 1\,048\,576$, the tree is rebuilt 17 times, after $i = 512; 1280; 2432; 4160; \dots; 671\,567; 1\,007\,861$. It turned out that the overall time for Algorithm 7 is not overly sensitive to the choice of the factor.

A more sophisticated strategy would take into account how much the tree has changed since the previous rebuild. To this end, we schedule the next rebuild to take place after $\varphi \cdot \text{numPoints}$ changes, where numPoints is the number of points in the tree during the current rebuild (i.e., $|\mathcal{S}_v|$ for its root node v) and φ is an adjustable parameter. Again, the overall time is not overly sensitive to the value of φ ; we chose $\varphi = 0.5$. To count “changes,” we keep track of the number of points that are inserted via Insert or deleted during NonDomPrune .

B. Maintaining Tight Bounds

Our rebuild routines generate trees with tight bounds in each node, i.e., $\text{lb}(\mathcal{S}_v) = \inf(\mathcal{S}_v)$ and $\text{ub}(\mathcal{S}_v) = \sup(\mathcal{S}_v)$ for all v . Since tighter bounds increase the probability of avoiding the costly case 4 in NonDomPrune , we want to preserve this property during the “ND-tree-mode” updates between the rebuilds.

The bounds of a node can change only if a point is added or removed (if v is a leaf) or if one of the children’s bounds changes. To implement this, another parameter updateLevel is added to NonDomPrune . On return, it indicates how much of the *parent’s* information should be recomputed, ranging

from UPDATE_NONE , to UPDATE_STATS (for statistical information, such as the number of points associated with a node), to UPDATE_ALL (also recompute bounds). Initialized to UPDATE_NONE , updateLevel is changed to UPDATE_ALL after deleting the subtree in case 2 (line 4 in Algorithm 2), after deleting a point in a leaf in line 11, or after removing an empty child in line 17. (Note that updateLevel can also be increased by the recursive calls in line 14.) To actually update the information in the current node, a function UpdateBounds is called if we reach the end of the “leaf” or “nonleaf” case (before line 12 or 20) with $\text{updateLevel} \neq \text{UPDATE_NONE}$. This function computes the required information (statistics only, or bounds as well) and may reduce updateLevel to UPDATE_STATS if lb and ub have not changed.

With corresponding minor changes to Insert and SplitLeaf , this keeps the bounds tight throughout.

C. Volume-Based Distance

One of the properties contributing to the effectiveness of SPND-trees is that the boxes of a node’s children are nonoverlapping. In order to not lose this property too quickly after a rebuild, we try to insert a new point into a child such that the overlap is not increased, or at least not by much.

More precisely, the distance of $\mathbf{p}_i$ to a subtree γ is determined as follows. We first compute the current volume of γ ’s box, $\text{vol}_{\text{current}} = \prod_{k=1}^D (\text{ub}(\gamma)(k) - \text{lb}(\gamma)(k))$, as well as vol_{new} , the volume of the box that would result from inserting $\mathbf{p}_i$ into γ , and let $\text{dist}(\mathbf{p}_i, \gamma) = \text{vol}_{\text{new}} - \text{vol}_{\text{current}} \geq 0$. This distance is zero whenever $\mathbf{p}_i$ is contained in γ ’s box.

However, after several ND-tree-based insertions, the boxes may overlap and, thus, the above $\text{dist}(\mathbf{p}_i, \gamma)$ may be zero for more than one child. To break ties, for $\text{dist} = 0$, this value is replaced by $-1/(\|\mathbf{p}_i - \mathbf{c}\|_2 + \epsilon)$, where $\mathbf{c} = (\text{lb}(\gamma) + \text{ub}(\gamma))/2$ is the center of the child’s box and ϵ is a small number to avoid division by zero if $\mathbf{p}_i = \mathbf{c}$. Thus, dist may return a negative value, which is the larger, the closer we are to the center of the box.

Overall, we insert $\mathbf{p}_i$ into the child with smallest $\text{dist}(\mathbf{p}_i, \gamma)$. This will not increase the overlap if $\text{dist} \leq 0$ and tries to at least keep the boxes small otherwise.

VI. NUMERICAL EXPERIMENTS

In this section, we want to assess the benefits of rebuilding for different dimensions, D , overall numbers of points, n , and geometric distributions of these. These properties are easiest controlled with synthetic data sets. For each n and each D , test sets with different characteristics have been generated.

CUBE: All coordinates $\mathbf{p}_i(k)$ are random numbers $r_{i,k}$ distributed uniformly in $[0, 1]$. The resulting points are therefore distributed uniformly in the unit cube.

SPHERE: Starting with the CUBE points, these are projected onto the unit sphere $\|\mathbf{y}\|_2 = 1$ by replacing each $\mathbf{p}_i$ with $\rho_i \cdot \mathbf{p}_i$, where $\rho_i = 1/\|\mathbf{p}_i\|_2$.

BALL: Here, the CUBE points are mapped into the unit ball $\|\mathbf{y}\|_2 \leq 1$ by scaling $\mathbf{p}_i$ with $\rho_i = \sqrt[D]{r_i}/\|\mathbf{p}_i\|_2$, where the r_i are distributed uniformly in $[0, 1]$.

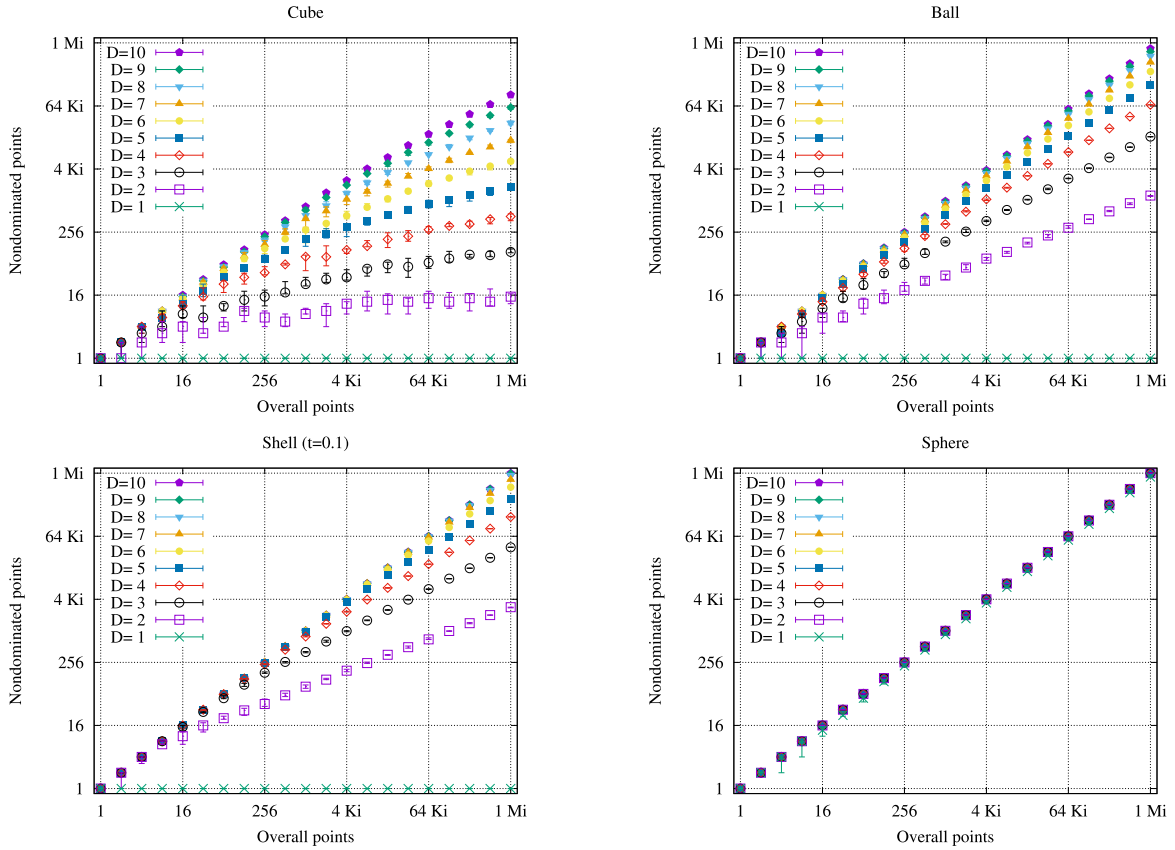


Fig. 3. Number of nondominated points in test sets with the distributions CUBE, BALL, SHELL, and SPHERE, different dimensions D , and up to $n = 1 \text{ Mi} = 2^{20} = 1\,048\,576$ points. For each combination (distribution, n , D), five test sets were generated, and the resulting minimum, median, and maximum number of nondominated points are represented by the error bars.

SHELL: The CUBE points are mapped into a shell of thickness t inside the unit sphere, $1 - t \leq \|\mathbf{y}\|_2 \leq 1$, by scaling $\mathbf{p}_i$ with $\rho_i = \sqrt[D]{(1 - r_i)(1 - t)^D + r_i} \|\mathbf{p}_i\|_2$. Choosing $t = 0$ or $t = 1$ reduces the SHELL distribution to SPHERE and BALL, respectively. In our experiments, we chose $t = 0.1$.

The CUBE sets put moderate demands on the algorithms because, in particular for small dimensions, the number of nondominated points grows slowly with n ; cf. Fig. 3. The SPHERE sets are very hard because all n points are nondominated. (In finite precision, this does not hold for $D = 1$ because not all $\mathbf{p}_i$ are exactly 1 after normalizing. However, this case is particularly challenging for ND-tree-based methods because all remaining points are identical and, therefore, all nodes and points in the tree must be inspected to verify that the next point is again nondominated.) The number of nondominated points in BALL is somewhere between CUBE and SPHERE, and SHELL is between BALL and SPHERE. For n large and D not too small, BALL can already be quite challenging.

The SHELL points imitate situations where the points under consideration are already close to the final Pareto set, e.g., in MOEAs [1].

The distribution of the points in SPHERE, BALL, and SHELL is slightly inhomogeneous w.r.t. the direction: the density is lowest along the axes and highest ($\sqrt{D}$ times higher) along the diagonal $\mathbf{y}(1) = \dots = \mathbf{y}(D)$. In contrast, (for BALL and SHELL), the density does not depend on the distance from

the origin. Note that for BALL a truly homogeneous distribution might be achieved by generating sufficiently many points according to CUBE and dropping those lying outside the unit ball. But then the “acceptance rate” $(\pi^{\frac{D}{2}} / \Gamma(D/2 + 1)) / 2^D$, i.e., the volume of the unit ball in relation to that of the enclosing cube $[-1, +1]^D$, would be very low for larger D [24] and, therefore, our approach is much more efficient. The resulting data sets are “harder” than homogeneous ones in the sense that they require more NonDomPrune calls, even if the Pareto archives may be slightly smaller.

For exploring extreme situations and checking correctness, we also used a fifth test set.

TRICKY: The points are taken cyclically from a discrete lattice with s^D points, $\mathbf{p}_i \in \{0, 1/s, \dots, s - 1/s\}^D$. (The last lattice point, $(s - 1/2, \dots, s - 1/s)$, is replaced with $(0, \dots, 0)$ to avoid having one point dominate all others.) In our experiments, we chose $s = 4$.

If $n > D$ then exactly D different points are nondominating (such points must have one component $\mathbf{p}_i(k) = s - 2/s$ and all other components $\mathbf{p}_i(k) = s - 1/s$), and if $n \gg s^D$ then these points occur with high multiplicity.

The numerical experiments were made on a single core of an Intel Core i7-4610M CPU running at 3.00 GHz in a laptop with 16-GiB main memory. The machine was not dedicated to these tests, so some fluctuations may be caused by the concurrent load. All timings include the sampling of diverse

TABLE I

TIMINGS (IN SECONDS) FOR TWO RUNS WITH $n = 1 \text{ Mi} = 1\,048\,576$ AND $D = 9$ BALL TEST SETS (714 305 AND 713 902 NONDOMINATED POINTS, RESP.) WITH DIFFERENT SETTINGS FOR THE TREE PARAMETERS, C AND P , REBUILDING METHODS (NO REBUILD, CLUSTERING-BASED, OR SPACE-PARTITIONED), UPDATING BOUNDS ACCORDING TO [15] OR TIGHTLY, AND CENTER- OR VOLUME-BASED DISTANCE TO SUBTREES

Rebuild	Bounds	dist2tree	1st run, $C/P =$			2nd run, $C/P =$		
			4/6	4/12	10/20	4/6	4/12	10/20
no	std	center	287.1	237.1	228.4	243.9	301.5	256.0
no	std	vol	330.7	316.4	387.6	358.1	332.1	379.9
no	tight	center	276.5	236.1	196.7	252.8	261.9	219.3
no	tight	vol	323.9	303.3	358.9	334.5	306.3	329.0
Cl	tight	center	211.9	198.4	207.8	210.1	211.3	228.3
Cl	tight	vol	187.6	177.5	181.3	175.5	182.2	196.1
SP	tight	center	75.0	73.0	96.5	79.3	75.6	99.6
SP	tight	vol	56.8	55.0	66.4	59.9	57.7	69.6

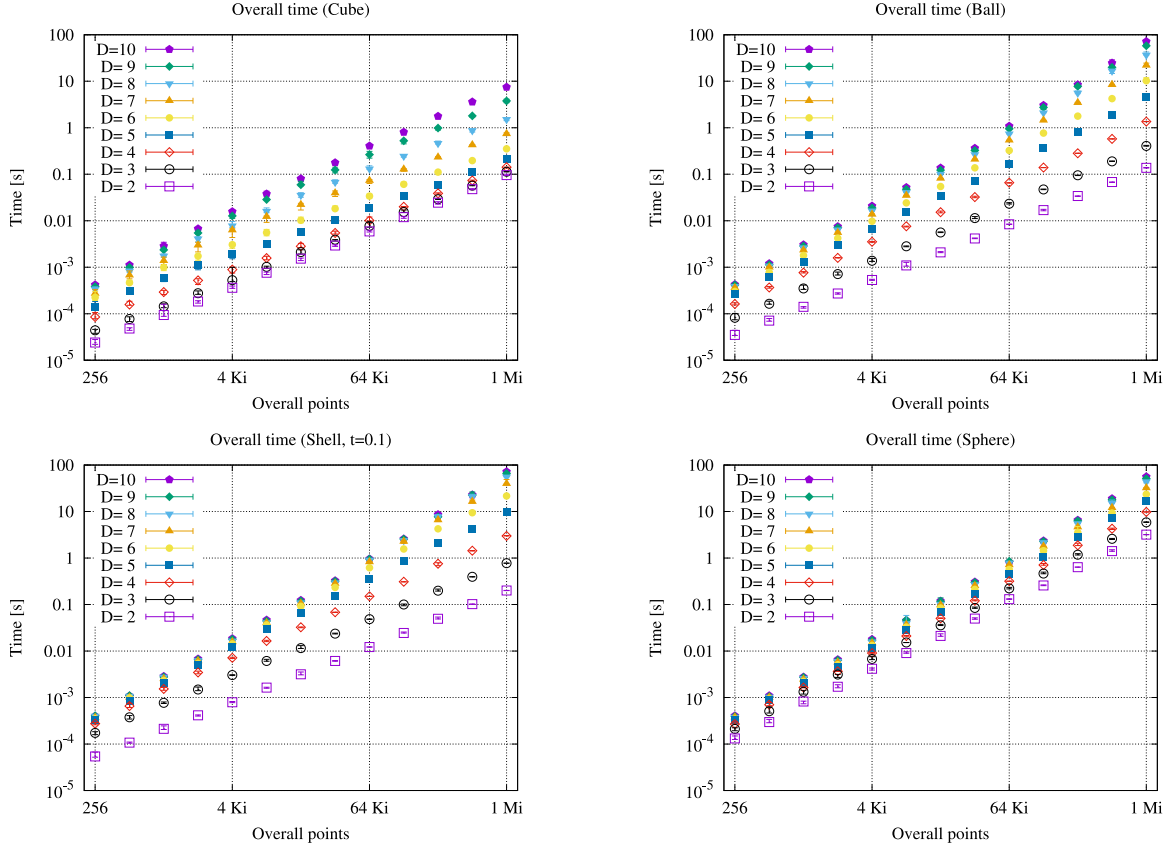


Fig. 4. Overall time (in seconds) for building the Pareto archives with SPND-trees ($C = 4$, $P = 12$, tight bounds, and volume-based distance) for the test sets from Fig. 3. For each combination (n, D) , the error bar shows the minimum, median, and maximum values over five test sets.

performance statistics, such as the number of dominance tests, etc. Without these, the timings would be roughly 5%–10% lower.

In Table I, we present detailed data for runs with two different BALL sets of size $n = 2^{20}$ and dimension $D = 9$. They exhibit some typical features observed also in other tests. With a “traditional” ND-tree according to [15], the tree parameters proposed by the authors ($C = D + 1$, $P = 20$) performed well, though not always optimally, over the whole range of tests. Therefore, we use these settings (boldface in the table, first row) as the base line in later comparisons. Without rebuilding, maintaining tight bounds may or may not reduce the overall time (the benefits tend to be larger for harder problems leading to larger trees), whereas the volume-based distance to subtrees almost always increased overall time. With

rebuilding, the situation changes. Clustering-based rebuilding with the steps-based strategy described in Section V-A tends to give noticeable speedup for higher dimensions, in particular, if combined with the volume-based distance for the updates between the rebuilds. For higher dimensions, by far, the largest improvement can be obtained with space-partitioned rebuilds, which again should be complemented with the volume-based distance. For this approach, $C = 4$ and $P = 12$ performed well in most cases (independently from D), so we use these settings (boldface in the table, last row) for the following timings and comparisons. (Note that our implementation requires tight bounds for both types of rebuilds).

In Fig. 4, we summarize the runs with SPND-trees for the four types of test sets (five runs for each type, size, and dimension), and Fig. 5 demonstrates that, for larger dimensions,

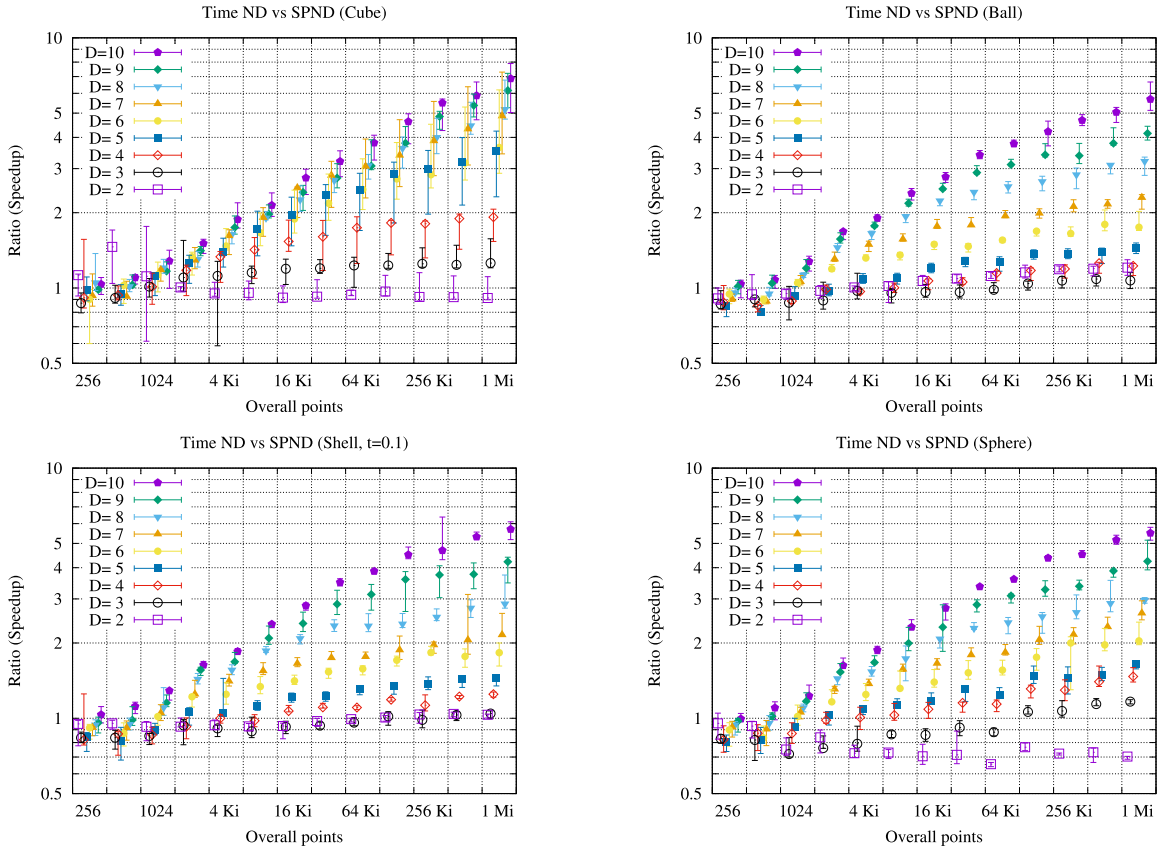


Fig. 5. Speedup by building the Pareto archives with SPND-trees ($C = 4$, $P = 12$, tight bounds, and volume-based distance), as compared to ND-trees ($C = D + 1$, $P = 20$, standard bounds, and center-based distance) for the test sets from Fig. 3. For each combination (n , D), the error bar shows the minimum, median, and maximum of the ratios t_{ND}/t_{SPND} over five test sets.

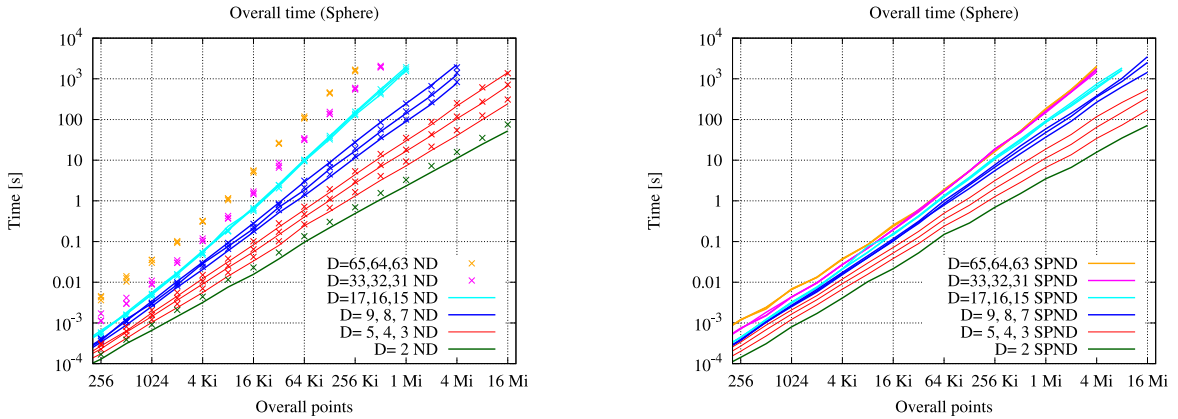


Fig. 6. Overall time (in seconds) for building the Pareto archives with ND-trees (left; solid lines: $C = D + 1$, $P = 20$; crosses: $C = 11$, $P = 20$) and SPND-trees (right; $C = 4$, $P = 12$) for SPHERE data.

significant speedup may be obtained, as compared to working with “traditional” ND-trees throughout. Note that the presentation is slightly different in Fig. 5; to ease distinguishing the error bars, the tests with the same n and different D have been spread horizontally, between two vertical dotted lines.

We want to mention that, for $D = 2$ and $D = 3$, superior methods with $\mathcal{O}(n \log n)$ overall complexity are known [20]; the timings for these dimensions are only shown for completeness.

In Fig. 6, the effect of dimensionality is exposed more clearly (recall that, with SPHERE data, all points are nondominated, independently from the dimension). In contrast to the preceding plots, each data point represents just one run. The default setting for ND-trees, $C = D + 1$ and $P = 20$, was feasible only for $D \leq 20$, due to the constraint $C \leq P + 1$; these runs are marked with solid lines. Therefore, we also used the fixed setting $C = 11$ and $P = 20$; these runs are marked with crosses and performed almost as well for $D \geq 7$. As the curves

TABLE II
NUMBER OF DOMINANCE TESTS ($<$, $\leq$, $>$, $\geq$) AND REAL COMPARISONS
($<$, $\leq$, $>$, $\geq$, $=$, $\neq$) IN BILLIONS FOR BUILDING THE PARETO
ARCHIVE FROM $n = 512$ K1 SPHERE POINTS

	ND-tree		SPND-tree	
	Dominance	Real	Dominance	Real
$D = 4$	0.48	0.91	0.30	0.75
$D = 8$	3.33	7.84	1.11	3.20
$D = 16$	24.90	71.61	1.84	7.92

for SPND-trees with $D = 31, 32, 33, 63, 64, 65$ almost coincide up to $n = 4$ Mi, dimensions exceeding a certain threshold seem to have a very limited impact on the overall time. This is not overly surprising because, even with $n = 4$ Mi $= 2^{22}$, only the first 22 directions are considered at all during rebuilds and, thus, the resulting tree depends only on these. It also does not make a significant difference whether D is a power of two (which leads to a “homogeneous” structure of the rebuilt trees) or close by (in this case, the lowest-level slicings within a node may be in different directions). The behavior may be slightly different with other distributions, but for larger D , the number of nondominated points typically is very close to n anyway.

Overall, the data suggest that with SPND-trees large Pareto archives can be handled even for high dimensions. Indeed, extrapolating the timings to $n = 8$ Mi and $n = 16$ Mi we estimate that adding a point to a Pareto archive of size $\approx 10^7$ and dimension $D = 65$ would take about 1 ms on average, including the amortized cost for the rebuilds.

Considering the number of dominance tests and the number of (real) comparisons reveal that SPND-trees require fewer dominance tests than ND-trees, but the decisions are slightly more expensive, taking roughly four versus three real comparisons per dominance test on average for $D = 16$; cf. Table II. (Note that in our C implementation of ND-trees and SPND-trees the dominance tests always return **TRUE** or **FALSE**, allowing for earlier return. In contrast, in [15], there is a single test returning the full information about dominance of both points w.r.t. the other, and therefore at most two dominance tests are required in **NonDomPrune**. Thus, our counts for dominance tests are twice as high.)

To assess the potential benefits for multiobjective evolutionary algorithms, we also used test sets obtained by running MOEA/D [5] (Python implementation from <https://pymoo.org> [25]) on the benchmark problem DTLZ1 [26] with $D = 4, 6, 8$ objectives, $q = 3D$ design variables, and default parameter settings. The focus of these experiments was on the time for building the archive, not on its percentage of overall optimization time, which would also depend on many details of the implementation, such as the programming language. Therefore, the (objective) points of each MOEA/D generation were dumped to file and then fed into our C implementation of ND/SPND-trees (with the standard settings indicated before).

The left part of Table III reports results for building an archive from the points of the first 500 generations. To avoid very large population sizes we used energy-based reference

directions [27]; the population size Π was set to $\binom{D+11}{D-1}/2^{D-3}$, one 2^{D-3} th of the population size that would result from using reference directions according to Das and Dennis [28] with $p = 12$ subdivisions. Using SPND-trees for building archives from these data sets gave speedups of more than two, as compared to standard ND-trees.

If one prefers to work with larger populations, one may use the Das/Dennis directions; then SPND-trees may even be beneficial for checking nondominance within the current population; see the right part of Table III ($p = 10$). For $n = \Pi = 286$, no speedup can be expected because the first rebuild would take place only after step 512 (so actually we use a standard ND-tree), but providing for rebuilds requires keeping lb and ub tight, which typically does not pay with ND-trees.

VII. CONCLUSION

ND-trees are a proven tool for efficiently building and updating Pareto archives, but their effectiveness may suffer if insertions and deletions lead to unbalanced trees or high overlap in the children’s boxes, and maintaining “good quality” of the tree over many operations may be impossible. We have proposed to instead restore this property from time to time by cheaply rebuilding the tree, either as an SPND-tree or with a clustering-based approach, making use of the fact that nondominance checks are superfluous because the points are already known to be mutually nondominated.

Numerical experiments have shown that in particular using SPND-trees, together with suitable strategies for updating the bounds lb and ub in each node (keep them tight) and for computing the distance of a new point to a subtree (volume-based) can lead to significant speedups, as compared to relying only on ND-tree-based updates. The benefits increase with the dimension, that is, the number of objectives in multiobjective optimization.

SPND-trees in a way combine the properties of higher-dimensional binary search trees (median-based bisection along one direction, cyclically), Quad-trees (condense multiple slicing directions into one node), and ND-trees (lb and ub for efficient nondominance checks); cf. Table IV. *Building* the trees by dimensional slicing ensures optimal balancing and nonoverlapping children’s boxes. Thereafter, the trees are *used* as plain ND-trees, relying on lb and ub.

Future investigations will reveal whether the performance can be improved further by more sophisticated strategies for scheduling rebuilds, based on changes in the tree, and how much additional speedup may be obtained by exploiting parallelism.

ACKNOWLEDGMENT

The author wants to thank Julius Bauß, Kathrin Klamroth, and Michael Stiglmayr from the Optimization Group for many fruitful discussions.

TABLE III

POPULATION SIZE (Π), OVERALL NUMBER OF POINTS (n), NUMBER OF NONDOMINATED POINTS ($|\mathcal{P}_n^N|$), TIME TAKEN WITH ND-TREES (T_{ND} , IN SECONDS) AND SPND-TREES (T_{SPND}), AND SPEEDUP ($S = T_{ND}/T_{SPND}$) FOR POINTS FROM RUNNING MOEA/D ON THE DTLZ1 BENCHMARK PROBLEM (TIMINGS AND SPEEDUPS ROUNDED TO THREE DIGITS)

D	Energy-based directions, building archive over 500 generations						Das/Dennis, single-generation nondominance-check				
	Π	n	$ \mathcal{P}_n^N $	T_{ND}	T_{SPND}	S	$\Pi = n$	$ \mathcal{P}_n^N $	T_{ND}	T_{SPND}	S
4	227	113,500	12,957	0.377	0.182	2.07	286	31	0.000094	0.000114	0.82
6	773	386,500	135,407	8.740	3.153	2.77	3,003	428	0.00779	0.00504	1.55
8	1,574	787,000	335,531	91.2	33.3	2.73	19,448	3,949	0.264	0.136	1.95

TABLE IV

PROPERTIES OF (MULTIDIMENSIONAL) BINARY SEARCH TREES, QUAD-TREES, ND-TREES, AND SPND-TREES: STORAGE LOCATION FOR THE POINTS, NUMBER OF CHILDREN PER NONLEAF NODE, HOW MANY ORTHOGONAL SLICINGS ARE DONE PER NODE?, AND WHAT DRIVES THE RECURSION IN NONDOMINANCE CHECKS?

	BST	Quad	ND	SPND
Points stored in	all	leaves	leaves	leaves
Children	2	$\approx 2^D$	$C \geq 2$	$C \geq 2$
Slicing directions per node	1	D	no slicing	$\log_2 C$
Recursion in search	ID-comparison	bit vector	lb, ub	lb, ub

REFERENCES

- [1] C. A. C. Coello, S. G. Brambila, J. F. Gamboa, M. G. C. Tapia, and R. H. Gómez, "Evolutionary multiobjective optimization: Open research areas and some challenges lying ahead," *Complex Intell. Syst.*, vol. 6, no. 1, pp. 221–236, 2020.
- [2] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [3] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength Pareto evolutionary algorithm," Eidgenössische Technische Hochschule Zürich (ETH), Institut für Technische Informatik und Kommunikationsnetze (TIK), Zürich, Switzerland, TIK-Rep. 103, 2001.
- [4] D. W. Corne, N. R. Jerram, J. D. Knowles, and M. J. Oates, "PESA-II: Region-based selection in evolutionary multiobjective optimization," in *Proc. 3rd Annu. Conf. Genet. Evol. Comput. (GECCO)*, 2001, pp. 283–290.
- [5] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.
- [6] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 577–601, Aug. 2014.
- [7] H. Jain and K. Deb, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part II: Handling constraints and extending to an adaptive approach," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 602–622, Aug. 2014.
- [8] J. E. Fieldsend, R. M. Everson, and S. Singh, "Using unconstrained elite archives for multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 7, no. 3, pp. 305–323, Jun. 2003.
- [9] X. Chen, "Pareto tree searching genetic algorithm: Approaching Pareto optimal front by searching Pareto optimal tree," Dept. Comput. Sci., Nankai Univ., Tianjin, China, Rep. NK-CS-2001–002, 2001.
- [10] S. Mostaghim, J. Teich, and A. Tyagi, "Comparison of data structures for storing Pareto-sets in MOEAs," in *Proc. Congr. Evol. Comput. (CEC)*, vol. 1. Honolulu, HI, USA, 2002, pp. 843–848.
- [11] S. Mostaghim and J. Teich, "Quad-trees: A data structure for storing Pareto sets in multiobjective evolutionary algorithms with elitism," in *Evolutionary Multiobjective Optimization* (Theoretical Advances and Applications), A. Abraham, L. Jain, and R. Goldberg, Eds. London, U.K.: Springer, 2005, pp. 81–104.
- [12] O. Schütze, "A new data structure for the nondominance problem in multi-objective optimization," in *Evolutionary Multi-Criterion Optimization*, (Lecture Notes in Computer Science 2632), C. M. Fonseca, P. J. Fleming, E. Zitzler, K. Deb, and L. Thiele, Eds. Berlin, Germany: Springer, 2003, pp. 509–518.
- [13] M. Drozdák, Y. Akimoto, H. Aguirre, and K. Tanaka, "Computational cost reduction of nondominated sorting using the M-front," *IEEE Trans. Evol. Comput.*, vol. 19, no. 5, pp. 659–678, Oct. 2015.
- [14] T. Glasmachers, "A fast incremental BSP tree archive for non-dominated points," in *Proc. 9th Int. Conf. Evol. Multi-Criterion Optim. (EMO)*, Münster, Germany, 2017, pp. 252–266.
- [15] A. Jaskiewicz and T. Lust, "ND-tree-based update: A fast algorithm for the dynamic nondominance problem," *IEEE Trans. Evol. Comput.*, vol. 22, no. 5, pp. 778–791, Oct. 2018.
- [16] J. E. Fieldsend, "Data structures for non-dominated sets: Implementations and empirical assessment of two decades of advances," in *Proc. Genet. Evol. Comput. Conf. (GECCO)*, New York, NY, USA, 2020, pp. 489–497.
- [17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [18] J. O'Rourke, *Computational Geometry in C*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 1998.
- [19] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Commun. ACM*, vol. 18, no. 9, pp. 509–517, Sep. 1975.
- [20] F. P. Preparata and M. I. Shamos, *Computational Geometry—An Introduction*. New York, NY, USA: Springer, 1985.
- [21] E. H. Cuthill and J. McKee, "Reducing the bandwidth of sparse symmetric matrices," in *Proc. 24th Nat. Conf. Assoc. Comput. Mach.*, 1969, pp. 157–172.
- [22] N. E. Gibbs, W. G. Poole, Jr., and P. K. Stockmeyer, "An algorithm for reducing the bandwidth and profile of a sparse matrix," *SIAM J. Numer. Anal.*, vol. 13, no. 2, pp. 236–250, Apr. 1976.
- [23] G. Malandain and J.-D. Boissonnat, "Computing the diameter of a point set," in *Discrete Geometry for Computer Imagery*, (Lecture Notes in Computer Science 2301), A. Braquelaire, J.-O. Lachaud, and A. Vialard, Eds. Berlin, Germany: Springer, 2002, pp. 197–208.
- [24] H. R. Parks, "The volume of the unit n -ball," *Math. Mag.*, vol. 86, no. 4, pp. 270–274, Oct. 2013.
- [25] J. Blank and K. Deb, "Pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89497–89509, 2020.
- [26] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable test problems for evolutionary multiobjective optimization," in *Evolutionary Multiobjective Optimization* (Theoretical Advances and Applications), A. Abraham, L. Jain, and R. Goldberg, Eds. London, U.K.: Springer, 2005, pp. 105–145.
- [27] J. Blank, K. Deb, Y. Dhebar, S. Bandaru, and H. Seada, "Generating well-spaced points on a unit simplex for evolutionary many-objective optimization," *IEEE Trans. Evol. Comput.*, vol. 25, no. 1, pp. 48–60, Feb. 2021.
- [28] I. Das and J. E. Dennis, "Normal-boundary intersection: A new method for generating the Pareto surface in nonlinear multicriteria optimization problems," *SIAM J. Optim.*, vol. 8, no. 3, pp. 631–657, Jul. 1998.

Bruno Lang received the Diploma degrees in mathematics and computer science and the Ph.D. degree from the University of Karlsruhe, Karlsruhe, Germany, in 1989, 1990, and 1991, respectively.

He is currently a Professor of Computer Science with the School of Mathematics and Natural Sciences, University of Wuppertal, Wuppertal, Germany. His research interests include efficient and parallel algorithms for problems in numerical linear algebra and optimization.